

claude-windows / 166d7059-d8ac-4fdd-a8c0-072cdaa7eccd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\166d7059-d8ac-4fdd-a8c0-072cdaa7eccd\subagents\workflows\wf_66c34d9d-057\agent-adedc93a4d1e1f14d.jsonl`
- SHA-256: `bd6128353f4795ec0d12ad6486f7a8ba6c95e1309903a255ac99ab0d99dd881d`
- Source modified: `2026-06-18T05:33:11+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_66c34d9d-057`
- session_id: `166d7059-d8ac-4fdd-a8c0-072cdaa7eccd`
```

Transcript

1. user (2026-06-18T05:31:43.299Z)

```
WORKTREE (cd here; code is origin/master f5f339a): C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/verify-n13
Run python as: cd "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/verify-n13" && PYTHONPATH=. python <your_script>
GOLDEN/manifest/features dir (ABSOLUTE ? pass this, the worktree output/ is empty):
C:/Users/avidu/Projects/badminton-highlight-indexer/output
```

```
REPRODUCE RECIPE (use the public API; write your own short script to scratch/lens_<name>.py):
import os
from dataclasses import replace
from backend.eval import rally_seg_eval as rse, eval_stats
from backend.eval.windowing import SERVED
from backend.eval.tolerance_metrics import over_seg_guard
OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"
golden = [g for g in rse.load_golden(os.path.join(OUT, "human_lovo_manifest.json"), OUT)
           if g["gts"] and os.path.exists(g["trajectory"])] # expect 13
baseline = SERVED # all windowing knobs OFF
cap12 = replace(SERVED, max_window_duration=12.0)
cap6 = replace(SERVED, max_window_duration=6.0)
milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,
                    inactivity_rally_thresh=0.20, inactivity_min_density=0.30)
rows = lambda p: {r["name"]: r for r in rse.evaluate(golden, p)["rows"]} # each row: tol_f1,
# f1, merge_rate, split_rate, merge_count, split_count, num_pred, num_gt, name,
dend_abs_median
# paired sign-test on a metric: eval_stats.paired_delta_ci([base[n][k] for n in
names], [cand[n][k] for n in names])
# returns {mean_delta, ci_low, ci_high, ci_excludes_zero,
sign_test: {wins, losses, ties, p_value}}
# over-seg guard: over_seg_guard(list(base_rows.values()), list(cand_rows.values()))
# returns {passes, strict_passes, base_net, cand_net, net_delta, no_merge_rate_regress,
merge_rate_regress: [...]}
# NOTE: rse.evaluate may print "R5 blob-fallback: forced ..." lines for the blob clip ? that
is the
# R5 logger and is HARMLESS/irrelevant here (the milestone does NOT enable blob_fallback).
Ignore it.
```

THE CLAIM TO FALSIFY: "The R1 milestone (W1 cap=6 + R4 inactivity-end=1.5/rally_thresh=0.20/min_density=0.30, no W2, no R5) still clears the net-over-seg promotion bar at n=13 ? a SIGNIFICANT R2 tolF1 win over baseline AND no honest over-seg regression ? so config-flip PR #204 still stands."

MY (the orchestrator's) measured n=13 numbers, for you to INDEPENDENTLY confirm or REFUTE ? do

NOT

trust them, recompute:

- baseline->milestone tolF1: ?+0.2222 [+0.1522,+0.2859], sign 12/0/1, p=0.0005
- cap12->cap6 tolF1: ?+0.1099 [+0.0703,+0.1526], sign 12/0/1, p=0.0005
- cap6->+R4 tolF1: ?+0.0397 [+0.0145,+0.0651], sign 9/1/3, p=0.0215 (ONE loss ? find it)
- mean merge_rate baseline->milestone: 0.651 -> 0.161; over_seg_guard NET PASS, no merge_rate regression
- new clips: GX010137 tolF1 0.448->0.685, GX010141 0.333->0.431 (both improve)

LENS = independent reproduction. Recompute the full ladder (baseline/cap12/cap6/milestone) aggregate mean tolF1 AND the three paired sign-tests (baseline->milestone, cap12->cap6, cap6->+R4) on all 13 golden clips, from your OWN short script using the public API. Confirm n==13 (no clip silently dropped). Report your computed ?/CI/sign(W/L/T)/p for each transition and whether they match mine within rounding. A transition "stands" iff its tolF1 CI excludes 0 AND its sign-test p<0.05. Flag ANY mismatch.

Return ONLY the structured verdict. Be precise with numbers.

2. assistant (2026-06-18T05:31:45.721Z)

I'll independently reproduce the ladder. Let me start by exploring the worktree structure and the public API.

3. user (2026-06-18T05:31:47.714Z)

- __init__.py
- __pycache__
- ablation.py
- annotations.py
- audio
- batch.py
- calibrate_local.py
- calibrate_wasb.py
- calibration.py
- classifier.py
- distill_local.py
- eval_partial_labels.py
- eval_stats.py
- experiment.py
- export_wasb_segments.py
- fusion_audio.py
- fusion_compare.py
- fusion_golden.py
- gemini_refine.py
- golden_fixtures.py
- gt_loader.py
- harness.py
- metrics.py
- per_user_eval.py
- per_user_gate.py
- per_user_regression.py
- promotion.py
- rally_seg_eval.py
- rally_seq_proto.py
- regression.py
- score_calibration.py
- segmentation_metrics.py
- serve_contrast.py
- served_gate_a.py
- splits.py

```
tolerance_metrics.py
training.py
windowing.py
---MANIFEST---
Badminton_BXH_2.rallies.csv
Badminton_BXH_2_golden_features.json
GX010125_highlights.mp4
GX010125_highlights_min5.mp4
GX010125_run
GX010128.rallies.csv
GX010128_golden_features.json
GX010137.rallies.csv
GX010137_golden_features.json
GX010141.rallies.csv
GX010141_golden_features.json
GX030094.rallies.csv
GX030094_golden_features.json
KushagraAviHighlight-Trailor.mp4
MahadevpuraSingles_highlights.mp4
REAL_FOOTAGE_VALIDATION_REPORT.md
ab_experiment_n6.json
adarsh_avi_full_vs_human.csv
adarsh_avi_singles_golden_features.json
adarsh_avi_vs_human.csv
adarsh_baseline_gated.csv
adarsh_baseline_segments.csv
adarsh_gemini_fullvideo.csv
adarsh_gemini_refined.csv
adarsh_gemini_smoke.csv
adarsh_min5s_segments.csv
adarsh_tuned_segments.csv
badminton_indexer.db
chunks
e2e_reel.mp4
```

4. assistant (2026-06-18T05:31:50.933Z)

Let me find the manifest file and confirm the golden set loads.

5. user (2026-06-18T05:31:52.630Z)

```
human_lovo_manifest.json
human_lovo_manifest.json.bak
local_calib_manifest.json
```

6. assistant (2026-06-18T05:31:54.982Z)

Good. Let me check the public API signatures to write a correct script.

7. user (2026-06-18T05:31:57.733Z)

```
101: def load_golden(manifest_path: str = DEFAULT_MANIFEST,
147: def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
---STATS---
90: def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
---WINDOWING---
6: deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED pipeline
14: SERVED preset is **single-sourced from the same Pydantic config defaults production reads**
20: The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
38:     "SERVED",
63:     max_window_duration: float = 0.0
64:     #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
69:     inactivity_end: float = 0.0
```

```

71:     inactivity_rally_thresh: float = 0.08
72:     inactivity_min_density: float = 0.34
74:     #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived blob
?
77:     blob_fallback: bool = False
83:     bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
92:         "max_window_duration": self.max_window_duration,
94:         "inactivity_end": self.inactivity_end,
95:         "inactivity_rally_thresh": self.inactivity_rally_thresh,
96:         "inactivity_min_density": self.inactivity_min_density,
97:         "blob_fallback": self.blob_fallback,
103:     """The SERVED windowing, lifted from the SAME Pydantic config defaults production reads.
111:     point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
120:         max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing (default
0=OFF)
122:         inactivity_end=float(idx.window_inactivity_end),      # R4 rally-end trim (default
0=OFF)
123:         inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
124:         inactivity_min_density=float(idx.inactivity_min_density),
125:         blob_fallback=bool(idx.window_blob_fallback),          # R5 blob-fallback (default
OFF)
132: SERVED: WindowingPreset = _served_preset_from_defaults()
136: #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
143: PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
147:     """The SERVED windowing a *specific* live config serves (overrides honoured).
152:     default in ``config.json`` ? the module-level :data:`SERVED` is the *default* operating
point;
159:         velocity_thresh=float(family_cfg.get("velocity_threshold", SERVED.velocity_thresh)),
160:         merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
161:         min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
162:         max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
163:         hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
164:         inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
165:         inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
SERVED.inactivity_rally_thresh)),
166:         inactivity_min_density=float(idx.get("inactivity_min_density",
SERVED.inactivity_min_density)),
167:         blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
168:         blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
169:         max_window_duration=preset.max_window_duration,

```

8. assistant (2026-06-18T05:32:03.117Z)

Let me check the `evaluate` return structure and the row keys, plus `paired\_delta\_ci` return shape.

9. user (2026-06-18T05:32:04.595Z)

```

147     def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
148                 iou: float = DEFAULT_IOU, min_crossings: int = 0,
149                 tau_start: Optional[float] = None, tau_end: Optional[float] = None) ->
Dict[str, Any]:
150         """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
151         rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block."""
152         rows: List[Dict[str, Any]] = []
153         for v in golden:
154             if not v["gts"] or not os.path.exists(v["trajectory"]):
155                 continue
156             preds = windows_for(v, preset, min_crossings)

```

```

157         row = score_intervals(preds, v["gts"], iou, tau_start, tau_end)
158         row["name"] = v["name"]
159         rows.append(row)
160     return {"preset": preset.to_dict(), "iou": iou, "min_crossings": min_crossings,
161           "rows": rows, "aggregate": aggregate(rows)}
162
163
164     def compare(golden: List[Dict[str, Any]], base: WindowingPreset, cand: WindowingPreset,
165               iou: float = DEFAULT_IOU, base_min_crossings: int = 0,
166               cand_min_crossings: int = 0,
167               tau_start: Optional[float] = None, tau_end: Optional[float] = None,
168               guard_weight_merge: Optional[float] = None,
169               guard_weight_split: Optional[float] = None,
170               guard_use_rates: bool = True) -> Dict[str, Any]:
171         """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
172 (+ CI +
173   sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
174 reports ? on the
175   over-segmentation metrics AND the **R1 net over-seg promotion guard**
176 (`over_seg_guard`):
177   the honest promote/block verdict for cand-over-base, with the strict per-video rule
178 reported
179   alongside so the refinement is auditable."""
180   from backend.eval import tolerance_metrics as tm
181   b = evaluate(golden, base, iou, base_min_crossings, tau_start, tau_end)
182   c = evaluate(golden, cand, iou, cand_min_crossings, tau_start, tau_end)
183   by_name_b = {r["name"]: r for r in b["rows"]}
184   paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
185   delta: Dict[str, Any] = {}
186   for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate",
187 "segment_count_ratio"):
188       delta[k] = eval_stats.paired_delta_ci([rb[k] for rb, _ in paired],
189                                             [rc[k] for _, rc in paired])
190   gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
191   if guard_weight_merge is not None:
192       gkw["weight_merge"] = guard_weight_merge
193   if guard_weight_split is not None:
194       gkw["weight_split"] = guard_weight_split
195   guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)
196   return {"base": b, "cand": c, "n_paired": len(paired), "delta": delta,
197 "over_seg_guard": guard}
198
199
200
201     # ----- #
202     # Reporting
203     # ----- #
204     def _ci(m: Optional[Dict[str, Any]]) -> str:
205         return "?" if not m else f"{m['mean']:.3f} [{m['ci_low']:.3f},{m['ci_high']:.3f}]"
206
207     def _format_guard(g: Dict[str, Any]) -> str:
208         """Render the R1 net over-seg promotion-guard verdict (with the strict rule for
209 contrast)."""
210         basis = "rate" if g["use_rates"] else "count"
211         lines = [f"\n## R1 over-seg promotion guard ({basis}-based, "
212               f"w_merge={g['weight_merge']}, w_split={g['weight_split']})",
213               f" NET verdict: {'PASS' if g['passes'] else 'BLOCK'} "
214               f"(net cost {g['base_net']:.3f} ? {g['cand_net']:.3f}, ?
215 {g['net_delta']:+.3f}; "
216               f"no_merge_rate_regress={g['no_merge_rate_regress']})",
217               f" strict per-video rule (R2 §2.3.3, for contrast): "
218               f"{'pass' if g['strict_passes'] else 'BLOCK'} "
219               f"(count increases ? merges:{len(g['new_merges'])})]
220 splits:{len(g['new_splits'])})"]
221         if g["merge_rate_regress"]:

```

```

213         names = ", ".join(f"{r['name']}({r['base']:.2f}?{r['cand']:.2f})"
214                             for r in g["merge_rate_regress"])
215         lines.append(f" ? merge_rate REGRESSED on: {names}")
216     return "\n".join(lines)
217
218
219     def format_report(result: Dict[str, Any]) -> str:
220         p = result["preset"]
221         bounded = []
222         if p.get("max_window_duration"):
223             bounded.append(f"cap={p['max_window_duration']}" + ("(hard)" if
p.get("hard_cap") else ""))
224         if p.get("inactivity_end"):
225             bounded.append(f"inact_end={p['inactivity_end']}/rt={p['inactivity_rally_thresh']}")
226             f"/md={p['inactivity_min_density']}")
227         if p.get("blob_fallback"):
228             bounded.append("blob_fallback")
229         extra = (" " + ", ".join(bounded)) if bounded else ""
230         lines = [f"# Rally-segmentation eval ? windowing '{p['name']}' "
231                 f"(vt={p['velocity_thresh']}, merge_gap={p['merge_gap']}, "
232                 f"min_win={p['min_window_duration']}{extra}), IoU={result['iou']}", ""]
233         lines.append("| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio |
pred/gt |")
234         lines.append("|---|---|---|---|---|---|---|")
235         for r in result["rows"]:
236             lines.append(f"| {r['name']} | {r['f1']:.3f} | {r['f1@0.25']:.3f} |
{r['f1@0.5']:.3f} | ")

```

10. assistant (2026-06-18T05:32:10.238Z)

Now let me check `paired\_delta\_ci` return shape and the row keys (tol_f1 vs merge_rate).

11. user (2026-06-18T05:32:11.918Z)

```

90     def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91                         confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
92                         eps: float = 0.0) -> Dict[str, Any]:
93         """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).
94
95         ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
96         by
97         fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
98         sign
99         test agrees ? not when a single mean ticked up.
100         """
101         n = min(len(a_values), len(b_values))
102         deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
103         summary = mean_with_ci(deltas, confidence, n_boot, seed)
104         return {
105             "mean_delta": summary["mean"],
106             "ci_low": summary["ci_low"],
107             "ci_high": summary["ci_high"],
108             "n": float(n),
109             "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
110             "sign_test": paired_sign_test(deltas, eps),
111         }
112
113     def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
114         """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
115         unit.
116         The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
117         first-seen (deterministic). Groups with no complement (a single group) yield no

```

```

fold.
117         """
118         seen: List[Any] = []
119         for g in groups:
120             if g not in seen:
121                 seen.append(g)
122         n = len(groups)
123         folds: List[Tuple[List[int], List[int]]] = []
124         for g in seen:
125             test = [i for i in range(n) if groups[i] == g]
126             train = [i for i in range(n) if groups[i] != g]
127             if test and train:
128                 folds.append((train, test))
129         return folds
130
131
132     def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133         """Out-of-fold predictions for honest stacking (C3).
134
135         For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
the
136         other groups and returns predictions for ``test_idx`` (in that order); results are
scattered back to a list aligned to the input items. So a *learned* producer's
137         outputs,
138         when used as arbiter features, never include in-sample predictions ? the leakage
path
139         that inflates measured lift on a tiny corpus. Items in a singleton group stay
``None``
140         (needs >= 2 groups to predict).
141         """
142         n = len(groups)
143         oof: List[Optional[Any]] = [None] * n
144         for train_idx, test_idx in grouped_folds(groups):
145             preds = fit_predict(train_idx, test_idx)
146             for pos, ti in enumerate(test_idx):
147                 if pos < len(preds):
148                     oof[ti] = preds[pos]
149         return oof
150

```

12. assistant (2026-06-18T05:32:14.890Z)

The note says the row keys use `tol\_f1` (with underscore). The reproduce recipe says `tol\_f1` and `r["name"]`. Now let me write my own verification script.

13. assistant (2026-06-18T05:32:30.120Z)

```

import os, json
from dataclasses import replace
from backend.eval import rally_seg_eval as rse, eval_stats
from backend.eval.windowing import SERVED
from backend.eval.tolerance_metrics import over_seg_guard

OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"

golden = [g for g in rse.load_golden(os.path.join(OUT, "human_lovo_manifest.json"), OUT)
          if g["gts"] and os.path.exists(g["trajectory"])]
print("N_GOLDEN =", len(golden))
print("CLIPS =", sorted(g["name"] for g in golden))

baseline = SERVED
cap12 = replace(SERVED, max_window_duration=12.0)
cap6 = replace(SERVED, max_window_duration=6.0)
milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,

```

```

        inactivity_rally_thresh=0.20, inactivity_min_density=0.30)

def rows(p):
    return {r["name"]: r for r in rse.evaluate(golden, p)["rows"]}

R_base = rows(baseline)
R_c12 = rows(cap12)
R_c6 = rows(cap6)
R_mile = rows(milestone)

names = sorted(R_base.keys())
assert set(names) == set(R_c12) == set(R_c6) == set(R_mile), "name set mismatch"
print("N_ROWS =", len(names))

def mean(d, k):
    return sum(d[n][k] for n in names) / len(names)

print("\n=== AGGREGATE MEAN tol_f1 ===")
for label, d in [("baseline", R_base), ("cap12", R_c12), ("cap6", R_c6), ("milestone", R_mile)]:
    print(f" {label:10s} tol_f1={mean(d, 'tol_f1'):.4f} merge_rate={mean(d, 'merge_rate'):.4f}")

def transition(name, A, B, key="tol_f1"):
    a = [A[n][key] for n in names]
    b = [B[n][key] for n in names]
    res = eval_stats.paired_delta_ci(a, b)
    st = res["sign_test"]
    print(f"\n-- {name} ({key}) ---")
    print(f" Delta={res['mean_delta']:+.4f} CI=[{res['ci_low']:+.4f},{res['ci_high']:+.4f}] "
          f"excl0={res['ci_excludes_zero']} sign W/L/T={st['wins']}/{st['losses']}/{st['ties']}")
    "
        f"p={st['p_value']:.4f}")
    stands = res["ci_excludes_zero"] and st["p_value"] < 0.05
    print(f" STANDS={stands}")
    return res, st

transition("baseline->milestone", R_base, R_mile)
transition("cap12->cap6", R_c12, R_c6)
res6, st6 = transition("cap6->R4(milestone)", R_c6, R_mile)

```

Find the ONE loss in cap6->milestone

```

print("\n=== cap6->R4 per-clip tol_f1 deltas ===")
for n in names:
    d = R_mile[n]["tol_f1"] - R_c6[n]["tol_f1"]
    tag = ""
    if d < -1e-9: tag = " <== LOSS"
    elif abs(d) < 1e-9: tag = " (tie)"
    print(f" {n:14s} {R_c6[n]['tol_f1']:.4f} -> {R_mile[n]['tol_f1']:.4f} d={d:+.4f}{tag}")

```

New clips

```

print("\n=== new clips baseline->milestone tol_f1 ===")
for n in ["GX010137", "GX010141"]:
    if n in R_base:
        print(f" {n}: {R_base[n]['tol_f1']:.4f} -> {R_mile[n]['tol_f1']:.4f}")

```

over_seg_guard baseline->milestone

```

g = over_seg_guard([R_base[n] for n in names], [R_mile[n] for n in names])
print("\n=== over_seg_guard baseline->milestone ===")
print(f" passes={g['passes']} strict={g['strict_passes']} base_net={g['base_net']:.4f} "
      f"and_net={g['and_net']:.4f} net_delta={g['net_delta']:+.4f} "
      f"no_merge_rate_regress={g['no_merge_rate_regress']}")
print(f" merge_rate_regress={g['merge_rate_regress']}")

```


File created successfully at: C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/verify-n13/scratch/lens_verify_n13.py (file state is current in your context ? no need to Read it back)

15. user (2026-06-18T05:32:40.974Z)

```
N_GOLDEN = 13
CLIPS = ['Badminton_BXH_2', 'GX010128', 'GX010137', 'GX010141', 'GX030094',
'adarsh_avi_singles', 'gbaaddy', 'kushagra_singles', 'largetest_doubles', 'mahadevpura_1',
'mahadevpura_2', 'mahadevpura_singles', 'testlarge_short']
N_ROWS = 13

=== AGGREGATE MEAN tol_f1 ===
baseline   tol_f1=0.1372  merge_rate=0.6513
cap12      tol_f1=0.2099  merge_rate=0.1726
cap6       tol_f1=0.3198  merge_rate=0.1726
milestone  tol_f1=0.3594  merge_rate=0.1610

--- baseline->milestone (tol_f1) ---
Delta=+0.2222 CI=[+0.1558,+0.2894] excl0=True sign W/L/T=12.0/0.0/1.0 p=0.0005
STANDS=True

--- cap12->cap6 (tol_f1) ---
Delta=+0.1099 CI=[+0.0700,+0.1522] excl0=True sign W/L/T=12.0/0.0/1.0 p=0.0005
STANDS=True

--- cap6->+R4(milestone) (tol_f1) ---
Delta=+0.0397 CI=[+0.0155,+0.0652] excl0=True sign W/L/T=9.0/1.0/3.0 p=0.0215
STANDS=True

=== cap6->+R4 per-clip tol_f1 deltas ===
Badminton_BXH_2 0.2273 -> 0.3279 d=+0.1006
GX010128        0.3472 -> 0.4662 d=+0.1189
GX010137        0.7273 -> 0.6849 d=-0.0423 <== LOSS
GX010141        0.3929 -> 0.4314 d=+0.0385
GX030094        0.3538 -> 0.4194 d=+0.0655
adarsh_avi_singles 0.4957 -> 0.5520 d=+0.0564
gbaaddy         0.1854 -> 0.2778 d=+0.0923
kushagra_singles 0.0741 -> 0.0779 d=+0.0038
largetest_doubles 0.5645 -> 0.5833 d=+0.0188
mahadevpura_1   0.0000 -> 0.0000 d=+0.0000 (tie)
mahadevpura_2   0.1220 -> 0.1220 d=+0.0000 (tie)
mahadevpura_singles 0.4667 -> 0.5301 d=+0.0635
testlarge_short 0.2000 -> 0.2000 d=+0.0000 (tie)

=== new clips baseline->milestone tol_f1 ===
GX010137: 0.4478 -> 0.6849
GX010141: 0.3333 -> 0.4314

=== over_seg_guard baseline->milestone ===
passes=True strict=False base_net=1.3026 cand_net=0.4289 net_delta=-0.8737
no_merge_rate_regress=True
merge_rate_regress=[]
```

16. assistant (2026-06-18T05:32:48.575Z)

All three transitions reproduce. Let me cross-check my numbers against the orchestrator's claims precisely.

My computed values vs orchestrator's:

```
| Transition | Mine ? / CI / sign / p | Orchestrator ? / CI / sign / p |
|---|---|---|
| baseline?milestone tolF1 | +0.2222 [+0.1558,+0.2894] 12/0/1 p=0.0005 | +0.2222
[+0.1522,+0.2859] 12/0/1 p=0.0005 |
```

```
| cap12?cap6 tolF1 | +0.1099 [+0.0700,+0.1522] 12/0/1 p=0.0005 | +0.1099 [+0.0703,+0.1526]
12/0/1 p=0.0005 |
| cap6?+R4 tolF1 | +0.0397 [+0.0155,+0.0652] 9/1/3 p=0.0215 | +0.0397 [+0.0145,+0.0651] 9/1/3
p=0.0215 |
```

Point estimates, sign-tests, and p-values match exactly. CI bounds differ only in the 3rd/4th decimal (bootstrap re-resampling jitter; all CIs still exclude 0 on both sides). The ONE loss in cap6?+R4 is GX010137 (0.7273?0.6849, d=?0.0423). merge_rate baseline?milestone 0.651?0.161 matches. over_seg_guard NET PASS (net_delta ?0.8737), no merge_rate regression. New clips GX010137 (0.448?0.685) and GX010141 (0.333?0.431) both improve, matching.

n=13 confirmed, no clip silently dropped. All three transitions stand (CI excludes 0 AND p<0.05).

17. user (2026-06-18T05:33:00.633Z)

Structured output provided successfully

18. assistant (2026-06-18T05:33:11.379Z)

Verification complete. All three transitions in the R1 milestone ladder independently reproduce at n=13 from my own script using the public API. The claim holds: every transition's tolF1 CI excludes 0 and its sign-test p<0.05, the net over-seg guard passes with no merge_rate regression, and PR #204 still stands. The only differences from the orchestrator's numbers are sub-0.004 bootstrap-CI jitter that does not change any verdict.

Key files:

- Script: `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/verify-n13/scratch/lens\_verify\_n13.py`
- Manifest/features: `C:/Users/avidu/Projects/badminton-highlight-indexer/output/human\_lovo\_manifest.json`